

LN15. 核方法 (续)

何 琨

计算机学院数据挖掘与机器学习实验室

华中科技大学 Hopcroft 计算科学研究中心

邮箱: brooklet60@hust.edu.cn

2026 年 4 月



- 1 良定义的核函数
 - 回顾: 最常见的核
 - 良定义核的概念
 - 定理
- 2 核机器
 - 实际应用
 - 核线性回归
 - 最近邻算法的核化
- 3 核 SVM
 - 原始 SVM
 - 核支持向量机-智能近邻

- 1 良定义的核函数
 - 回顾: 最常见的核
 - 良定义核的概念
 - 定理

- 2 核机器
 - 实际应用
 - 核线性回归
 - 最近邻算法的核化

- 3 核 SVM
 - 原始 SVM
 - 核支持向量机-智能近邻

最常见的核函数:

- 线性核函数: $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$
- RBF: $k(\mathbf{x}, \mathbf{z}) = e^{-\frac{(\mathbf{x}-\mathbf{z})^2}{\sigma^2}}$
- 多项式核函数: $k(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}^\top \mathbf{z})^d$

定义

递归地使用一至多次如下规则，得到的核称为良定义的核。

- ① $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$
- ② $k(\mathbf{x}, \mathbf{z}) = ck_1(\mathbf{x}, \mathbf{z})$
- ③ $k(\mathbf{x}, \mathbf{z}) = k_1(\mathbf{x}, \mathbf{z}) + k_2(\mathbf{x}, \mathbf{z})$
- ④ $k(\mathbf{x}, \mathbf{z}) = g(k_1(\mathbf{x}, \mathbf{z}))$
- ⑤ $k(\mathbf{x}, \mathbf{z}) = k_1(\mathbf{x}, \mathbf{z})k_2(\mathbf{x}, \mathbf{z})$
- ⑥ $k(\mathbf{x}, \mathbf{z}) = f(\mathbf{x})k_1(\mathbf{x}, \mathbf{z})f(\mathbf{z})$
- ⑦ $k(\mathbf{x}, \mathbf{z}) = e^{k_1(\mathbf{x}, \mathbf{z})}$
- ⑧ $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{A} \mathbf{z}$

其中， k_1, k_2 是良定义的核函数， $c \geq 0$ ， g 是系数为正的多项式函数， f 是任意函数， $\mathbf{A} \succeq 0$ 是半正定矩阵。

核函数是良定义的等价于相应的核矩阵 K 是半正定的，即等价于以下任一种表述：

- ① K 的所有特征值都是非负的
- ② $\exists$ 实矩阵 P , s.t. $K = P^\top P$
- ③ $\forall$ 实向量 $\mathbf{x}$, $\mathbf{x}^\top K \mathbf{x} \geq 0$

证明具有整数 d 的多项式核和线性核都是良定义的核函数是很容易的。

定理 1

定理 1

RBF 核函数 $k(\mathbf{x}, \mathbf{z}) = e^{\frac{-(\mathbf{x}-\mathbf{z})^2}{\sigma^2}}$ 是一个良定义的核。

定理 1

定理 1

RBF 核函数 $k(\mathbf{x}, \mathbf{z}) = e^{\frac{-(\mathbf{x}-\mathbf{z})^2}{\sigma^2}}$ 是一个良定义的核。

测试 1

尝试给出证明。

参考：

- ① $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$
- ② $k(\mathbf{x}, \mathbf{z}) = ck_1(\mathbf{x}, \mathbf{z}), \quad c \geq 0$
- ③ $k(\mathbf{x}, \mathbf{z}) = k_1(\mathbf{x}, \mathbf{z}) + k_2(\mathbf{x}, \mathbf{z}), \quad k_1, k_2$ 是良定义的核函数
- ④ $k(\mathbf{x}, \mathbf{z}) = g(k_1(\mathbf{x}, \mathbf{z})), g$ 是系数为正的多项式函数
- ⑤ $k(\mathbf{x}, \mathbf{z}) = k_1(\mathbf{x}, \mathbf{z})k_2(\mathbf{x}, \mathbf{z})$
- ⑥ $k(\mathbf{x}, \mathbf{z}) = f(\mathbf{x})k_1(\mathbf{x}, \mathbf{z})f(\mathbf{z}), f$ 是任意函数
- ⑦ $k(\mathbf{x}, \mathbf{z}) = e^{k_1(\mathbf{x}, \mathbf{z})}$
- ⑧ $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{A} \mathbf{z}, \mathbf{A} \succeq 0$ 是半正定矩阵

定理 1 的证明

Proof.

$$k_1(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z} \quad \text{well defined by rule 1}$$

$$k_2(\mathbf{x}, \mathbf{z}) = \frac{2}{\sigma^2} k_1(\mathbf{x}, \mathbf{z}) = \frac{2}{\sigma^2} \mathbf{x}^\top \mathbf{z} \quad \text{well defined by rule 2}$$

$$k_3(\mathbf{x}, \mathbf{z}) = e^{k_2(\mathbf{x}, \mathbf{z})} = e^{\frac{2\mathbf{x}^\top \mathbf{z}}{\sigma^2}} \quad \text{well defined by rule 7}$$

$$k_4(\mathbf{x}, \mathbf{z}) = e^{-\frac{\mathbf{x}^\top \mathbf{x}}{\sigma^2}} k_3(\mathbf{x}, \mathbf{z}) e^{-\frac{\mathbf{z}^\top \mathbf{z}}{\sigma^2}} = e^{-\frac{\mathbf{x}^\top \mathbf{x}}{\sigma^2}} e^{\frac{2\mathbf{x}^\top \mathbf{z}}{\sigma^2}} e^{-\frac{\mathbf{z}^\top \mathbf{z}}{\sigma^2}}$$

well defined by rule 6 with $f(\mathbf{x}) = e^{-\frac{\mathbf{x}^\top \mathbf{x}}{\sigma^2}}$

$$= e^{\frac{-\mathbf{x}^\top \mathbf{x} + 2\mathbf{x}^\top \mathbf{z} - \mathbf{z}^\top \mathbf{z}}{\sigma^2}} = e^{\frac{-(\mathbf{x} - \mathbf{z})^\top (\mathbf{x} - \mathbf{z})}{\sigma^2}} = k_{RBF}(\mathbf{x}, \mathbf{z})$$

定理 2

定义在任意两个集合 $S_1, S_2 \subseteq \Omega$ 上的如下函数为核函数:

$$k(S_1, S_2) = e^{|S_1 \cap S_2|}.$$

定理 2

定理 2

定义在任意两个集合 $S_1, S_2 \subseteq \Omega$ 上的如下函数为核函数:

$$k(S_1, S_2) = e^{|S_1 \cap S_2|}.$$

测试 2

尝试给出证明! 参考:

- ① $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$
- ② $k(\mathbf{x}, \mathbf{z}) = ck_1(\mathbf{x}, \mathbf{z}), \quad c \geq 0$
- ③ $k(\mathbf{x}, \mathbf{z}) = k_1(\mathbf{x}, \mathbf{z}) + k_2(\mathbf{x}, \mathbf{z}), \quad k_1, k_2$ 是良定义的核函数
- ④ $k(\mathbf{x}, \mathbf{z}) = g(k_1(\mathbf{x}, \mathbf{z})), g$ 是系数为正的多项式函数
- ⑤ $k(\mathbf{x}, \mathbf{z}) = k_1(\mathbf{x}, \mathbf{z})k_2(\mathbf{x}, \mathbf{z})$
- ⑥ $k(\mathbf{x}, \mathbf{z}) = f(\mathbf{x})k_1(\mathbf{x}, \mathbf{z})f(\mathbf{z}), f$ 是任意函数
- ⑦ $k(\mathbf{x}, \mathbf{z}) = e^{k_1(\mathbf{x}, \mathbf{z})}$
- ⑧ $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{A} \mathbf{z}, \mathbf{A} \succeq 0$ 是半正定矩阵

定理 2 证明

Proof.

列出所有可能的样本 Ω ，并将它们排列成一个有序的列表。我们定义一个向量 $\mathbf{x}_S \in \{0, 1\}^{|\Omega|}$ ，其中它的每个元素都表示对应的样本是否包含在集合 S 中。

证明这一点很容易：

测试 2

如何证明？参考：

- 1 $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$
- 2 $k(\mathbf{x}, \mathbf{z}) = ck_1(\mathbf{x}, \mathbf{z}), \quad c \geq 0$
- 3 $k(\mathbf{x}, \mathbf{z}) = k_1(\mathbf{x}, \mathbf{z}) + k_2(\mathbf{x}, \mathbf{z}), \quad k_1, k_2$ 是良定义的核函数
- 4 $k(\mathbf{x}, \mathbf{z}) = g(k_1(\mathbf{x}, \mathbf{z})), g$ 是系数为正的多项式函数
- 5 $k(\mathbf{x}, \mathbf{z}) = k_1(\mathbf{x}, \mathbf{z})k_2(\mathbf{x}, \mathbf{z})$
- 6 $k(\mathbf{x}, \mathbf{z}) = f(\mathbf{x})k_1(\mathbf{x}, \mathbf{z})f(\mathbf{z}), f$ 是任意函数
- 7 $k(\mathbf{x}, \mathbf{z}) = e^{k_1(\mathbf{x}, \mathbf{z})}$
- 8 $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{A} \mathbf{z}, \mathbf{A} \succeq 0$ 是半正定矩阵

定理 2 证明

Proof.

列出所有可能的样本 Ω ，并将它们排列成一个有序的列表。我们定义一个向量 $\mathbf{x}_S \in \{0, 1\}^{|\Omega|}$ ，其中它的每个元素都表示对应的样本是否包含在集合 S 中。
证明这一点很容易：

$$k(S_1, S_2) = e^{\mathbf{x}_{S_1}^\top \mathbf{x}_{S_2}},$$

它是由规则 1 和规则 7 生成的良定义核函数。

- 1 良定义的核函数
 - 回顾: 最常见的核
 - 良定义核的概念
 - 定理

- 2 核机器
 - 实际应用
 - 核线性回归
 - 最近邻算法的核化

- 3 核 SVM
 - 原始 SVM
 - 核支持向量机-智能近邻

(在实践中) 对一个算法进行核化 (Kernelized) 可以分为 3 个步骤:

- 1 证明解存在于训练点张成的空间内 (如 $\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{x}_i$, 对于某些参数 α_i)
- 2 重写算法和分类器, 使所有训练或测试输入 $\mathbf{x}_i$ 只能通过与其他输入的内积访问, 例如 $\mathbf{x}_i^\top \mathbf{x}_j$
- 3 定义一个核函数, 用 $k(\mathbf{x}_i, \mathbf{x}_j)$ 代替 $\mathbf{x}_i^\top \mathbf{x}_j$ 。

朴素的最小二乘法 (OLS) [也称为线性回归], 最小化以下均方损失

$$\min_{\mathbf{w}} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2,$$

目标是找到超平面 $\mathbf{w}$ 。在测试点上的预测为 $h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ 。

设 $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]$, $\mathbf{y} = [y_1 \dots, y_n]^\top$, 则最小二乘法的闭式解为:

$$\mathbf{w} = (\mathbf{X}\mathbf{X}^\top)^{-1}\mathbf{X}\mathbf{y} \quad (1)$$

首先，将解 $\mathbf{w}$ 表示为训练输入的线性组合

$$\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{x}_i = \mathbf{X} \vec{\alpha}.$$

从前面的章节可知，若对方程1用梯度更新寻找最小值，向量 $\vec{\alpha}$ 必然始终存在。

初始向量设置为 $\mathbf{w}_0 = \vec{0}$ (因为均方损失是凸的，解与初始值无关)。

同样地，在测试时，测试点只能通过与训练输入点的内积来访问：

$$h(\mathbf{z}) = \mathbf{w}^\top \mathbf{z} = \sum_{i=1}^n \alpha_i \mathbf{x}_i^\top \mathbf{z}.$$

我们现在可以将算法核化，将任何内积 $\mathbf{x}^\top \mathbf{z}$ 替换为 $k(\mathbf{x}, \mathbf{z})$ 。

还需要说明的是，我们也可以用闭式解给出 α 的值。这个可以很容易就能推出。

定理

核化的朴素最小二乘法有解 $\vec{\alpha} = \mathbf{K}^{-1}\mathbf{y}$

定理

核化的朴素最小二乘法有解 $\vec{\alpha} = \mathbf{K}^{-1}\mathbf{y}$

Proof.

$$\begin{aligned}\mathbf{X}\vec{\alpha} &= \mathbf{w} = (\mathbf{X}\mathbf{X}^\top)^{-1}\mathbf{X}\mathbf{y} \quad | \text{通过左乘 } \mathbf{X}^\top\mathbf{X}\mathbf{X}^\top \\ (\mathbf{X}^\top\mathbf{X})(\mathbf{X}^\top\mathbf{X})\vec{\alpha} &= \mathbf{X}^\top(\mathbf{X}\mathbf{X}^\top(\mathbf{X}\mathbf{X}^\top)^{-1})\mathbf{X}\mathbf{y} \quad | \text{替换 } \mathbf{K} = \mathbf{X}^\top\mathbf{X} \\ \mathbf{K}^2\vec{\alpha} &= \mathbf{K}\mathbf{y} \quad | \text{左乘 } (\mathbf{K}^{-1})^2 \\ \vec{\alpha} &= \mathbf{K}^{-1}\mathbf{y}\end{aligned}$$

核回归可以扩展到岭回归的核化版本。解就变为

$$\vec{\alpha} = (\mathbf{K} + \tau^2 \mathbf{I})^{-1} \mathbf{y}.$$

在实际中，较小的 $\tau^2 > 0$ 值可以增加稳定性，特别是 $\mathbf{K}$ 不可逆的情况。

当 $\tau = 0$ ，则“核岭回归”退化为核化朴素最小二乘法。典型的核化岭回归也称为核回归。

注意，我们定义了 $\mathbf{w} = \mathbf{X}\vec{\alpha}$ 。测试点 $\mathbf{z}$ 的预测就变成了

$$h(\mathbf{z}) = \mathbf{z}^\top \mathbf{w} = \mathbf{z}^\top \underbrace{\mathbf{X}\vec{\alpha}}_{\mathbf{w}} = \underbrace{\mathbf{k}_*}_{\mathbf{z}^\top \mathbf{X}} \underbrace{(\mathbf{K} + \tau^2 \mathbf{I})^{-1} \mathbf{y}}_{\vec{\alpha}} = \mathbf{k}_* \vec{\alpha},$$

或者，如果采用闭式解的形式：

$$h(\mathbf{z}) = \mathbf{k}_* (\mathbf{K} + \tau^2 \mathbf{I})^{-1} \mathbf{y},$$

其中 $\mathbf{k}_*$ 是测试点与训练点的核 (向量)，即第 i 维对应 $[\mathbf{k}_*]_i = \phi(\mathbf{z})^\top \phi(\mathbf{x}_i)$ ，测试点 $\mathbf{z}$ 与训练点 $\mathbf{x}_i$ 通过 ϕ 映射到特征空间后的内积。

测试

设 $D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$.

如何将最近邻 (欧几里得距离) 算法核函数化?

- ① 良定义的核函数
 - 回顾: 最常见的核
 - 良定义核的概念
 - 定理
- ② 核机器
 - 实际应用
 - 核线性回归
 - 最近邻算法的核化
- ③ 核 SVM
 - 原始 SVM
 - 核支持向量机-智能近邻

原始的 SVM 是一个二次规划问题:

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \mathbf{w}^\top \mathbf{w} + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & \forall i, \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i \\ & \xi_i \geq 0 \end{aligned}$$

它具有对偶形式

$$\begin{aligned} \min_{\alpha_1, \dots, \alpha_n} \quad & \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K_{ij} - \sum_{i=1}^n \alpha_i \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C \\ & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

这里, $\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \phi(\mathbf{x}_i)$ (尽管从未计算) 和

$$h(\mathbf{x}) = \text{sign} \left(\sum_{i=1}^n \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b \right).$$

支持向量

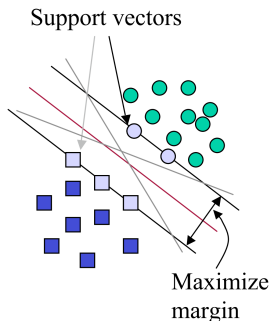
在支持向量方面，对偶问题有一个很好的解释：

对于原始公式，我们知道只有支持向量满足等式约束：

$$y_i(\mathbf{w}^\top \phi(x_i) + b) = 1.$$

在对偶形式中，这些相同的训练输入可以被识别为它们对应的满足 $\alpha_i > 0$ 的对偶值（所有其他训练输入都有 $\alpha_i = 0$ ）。

在测试时，只需要计算 $h(x)$ 对支持向量的和，所有对应 $\alpha_i = 0$ 的输入 x_i 可以在训练时忽略。



对偶版本的一个明显问题是 b 不再是优化的一部分。但是，我们需要它来进行分类。幸运的是，我们知道原始问题的解和对偶解是相同的。在对偶版本中，支持向量是那些 $\alpha_i > 0$ 的向量。因此，我们可以解出 b 的方程：

$$\begin{aligned}y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) &= 1 \\y_i \left(\sum_j y_j \alpha_j k(\mathbf{x}_j, \mathbf{x}_i) + b \right) &= 1 \\y_i - \sum_j y_j \alpha_j k(\mathbf{x}_j, \mathbf{x}_i) &= b\end{aligned}$$

这允许我们从支持向量中求解 b (在实际中，最好从几个支持向量中求 b 的平均值，因为可能存在数值精度问题)。

思考

硬间隔支持向量机的对偶形式是什么？

回顾：k 近邻算法

还记得 k 近邻算法吗？对于二元分类问题 ($y_i \in \{+1, -1\}$)，我们可以将测试点 $\mathbf{z}$ 的决策函数写为

$$h(\mathbf{z}) = \text{sign} \left(\sum_{i=1}^n y_i \delta^{nn}(\mathbf{x}_i, \mathbf{z}) \right),$$

$\delta^{nn}(\mathbf{z}, \mathbf{x}_i) \in \{0, 1\}$, 当 $\mathbf{x}_i$ 是测试点 $\mathbf{z}$ 的 k 个最近邻居之一时, $\delta^{nn}(\mathbf{z}, \mathbf{x}_i) = 1$ 。

KNN 决策边界:

$$h(\mathbf{z}) = \text{sign} \left(\sum_{i=1}^n y_i \delta^{nn}(\mathbf{x}_i, \mathbf{z}) \right)$$

SVM 决策边界:

$$h(\mathbf{z}) = \text{sign} \left(\sum_{i=1}^n y_i \alpha_i k(\mathbf{x}_i, \mathbf{z}) + b \right)$$

与 KNN 的非常相似，但它不是将决策限制在 k 个最近的邻居，而是考虑所有的训练点，但核函数将更多的权重分配给那些更接近的点 (大的 $k(\mathbf{z}, \mathbf{x}_i)$)。

从某种意义上说，可以把 RBF 核看作是一个软的最近邻赋值，因为随着距离的指数衰减几乎不会给 $\mathbf{z}$ 的所有相邻点赋值。

SVM 核化算法还为每个训练点学习了权重 $\alpha_i > 0$ 和偏差 b ，本质上，它通过设置许多 $\alpha_i = 0$ 来“移除”无用的训练点。

The End